Lab 4 - API & Data

Reminder: One goal of the lab is to get a feel for how teams work on a codebase together. For deeper mastery, consider completing this lab individually as well, but it is not required. You may also want to consider coding together in a Zoom session.

This lab builds on Lab 3 as we continue to close issues in our project backlog. The same flow from Lab 3 applies here:

- Pull changes from GitHub to your local machine
- Create the feature branch
- Create the code
- Verify that it works (!)
- Push the feature branch to Github
- Open a pull request
- Review pull request code with your team
- Have a teammate merge the code to main.

I will not call these steps out in Lab 4, these are part of your development workflow. Again, there will be some elements of this lab that test your resolve- 'find your way' as a team.

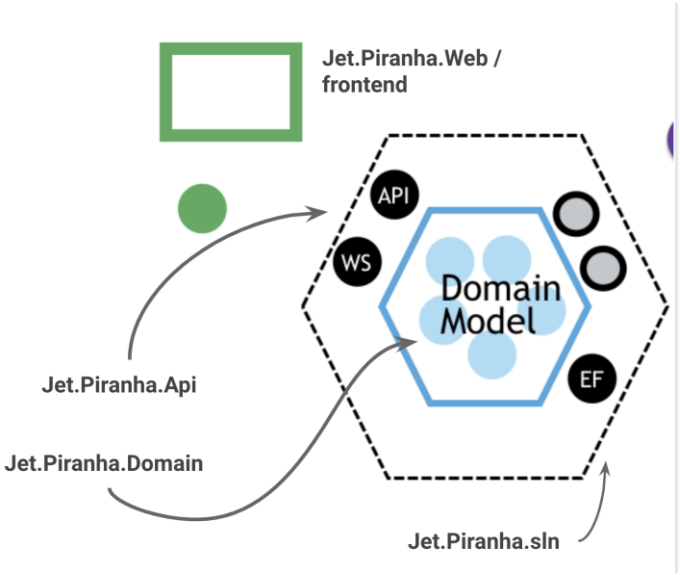
Make sure you have the REST Client extension installed in VS Code (see workshop 4).

Part 1: Building an API

1. API Project Structure

- Clone the API repo from your org's Github (see Lab 3)
- Inside the root folder (e.g "Jet-Piranha-Api") , create the main solution file
    - Create the main solution file by running the following command: **`dotnet new sln -n YOUR-PROJECT-NAME`** `(e.g. Jet.Piranha)`

Review the lecture notes from 4.1 and 4.2. Recall that we create a domain model project to hold all of the business objects and logic, and a separate project for our API. This is an extended version of the "Single Responsibility Pattern": the API handles the HTTP Request / Response and the Domain handle pure business logic.



- Create the domain project:
  - **`dotnet new classlib -n YOUR-PROJECT-NAME.Domain`**
- Create the API project:
  - **`dotnet new webapi -n YOUR-PROJECT-NAME.Api`**

The purpose of the solution file is to be a container for all of our projects. When we run 'dotnet build' or 'dotnet restore', the .NET framework will find the solution file and run these commands for **every** child project. Let's add the two new projects to our solution.

- Inside the root (i.e., your project) folder, add projects to solution files. (Tip: you can run (`ls **/*.csproj`) to find exact path of the `csproj` files.
  - **`dotnet sln add YOUR-PROJECT-NAME.Api/YOUR-PROJECT-NAME.Api.csproj`**
  - **`dotnet sln add YOUR-PROJECT-NAME.Domain/YOUR-PROJECT-NAME.Domain.csproj`**
- Verify that things are working by building the solution
  - **`dotnet restore && dotnet build (MacOS)`**
  - **`dotnet restore ; dotnet build (Windows PowerShell)`**

The API will use classes in the Domain so it will need a project reference to it. Let's add a project reference from API to Domain:

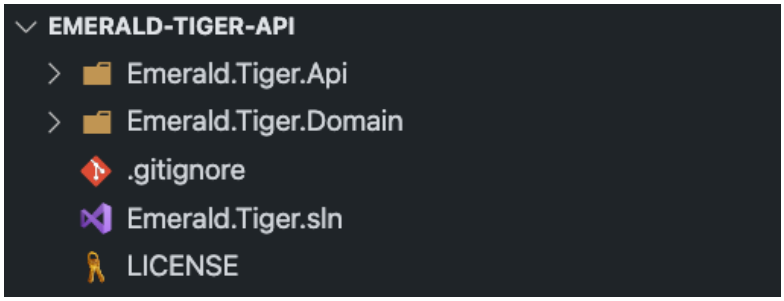
- Inside the root folder:
    - **`dotnet add YOUR-PROJECT-NAME.Api/YOUR-PROJECT-NAME.Api.csproj reference YOUR-PROJECT-NAME.Domain/YOUR-PROJECT-NAME.Domain.csproj`**
    - **Hint**: after typing '**`dotnet add`**', enter the first letter of your project name and hit **`TAB-`** the terminal will help you complete the project and file names.
- Verify that things are working by building the solution
    - **`dotnet restore && dotnet build`**
      `(use semicolon in place of && in Windows PS)`

We need to clean a few things up before pushing things to GitHub.

- Inside the API project folder:
    - Remove **`Program.cs`** file.
    - Add **`App.cs`** provided in Carmen (under Files/Labs folder)
    - Add **`Startup.cs`** provided in Carmen (under: Files/Labs folder)
- Inside the Domain project folder:
    - Remove the **Class1.cs** sample code
- Inside the project root create a .**gitignore** file
    - Run the command **`dotnet new gitignore`**
- Verify things are working by building the solution
    - **`dotnet restore && dotnet build`**

Push the changes to GitHub, following the push and pull request workflow. Be sure to link the push and PR to the issue number for creating the API structure.

Hint: when you are done, the project layout should look like this (you may not have a LICENSE file, that's okay):



2. Catalog API - Get All Items

Pull the code from **1. API Structure** in Github to your local machine; this step should be done on a **Catalog** branch.

- Create folder in the **Domain** project called **Catalog**
- Create a new business entity to represent an item in the catalog:
    - Inside the **Catalog** folder, create **Item.cs:**

```
using System;

namespace Jet.Piranha.Domain.Catalog
{
    public class Item
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Description { get; set; }
        public string Brand { get; set; }
        public decimal Price { get; set; }

    }
}
```

If we look at the data for the catalog in the frontend, we notice that the concept of rating is missing. Let's create an object for holding a Rating on an Item, along with some trivial business logic.

- Create a `class` for ratings called **Rating.cs**:

```
namespace Emerald.Tiger.Domain.Catalog;

public class Rating
{
    public int Stars { get; set; }
    public string UserName { get; set; }
    public string Review { get; set; }
}
```

- A rating must have valid stars between 1 and 5; also reviews can be blank, but the user name cannot. Add a constructor to enforce the logic inside the **Rating** `class`:

```
public Rating(int stars, string userName, string review)
{
    if(stars < 1 || stars > 5)
    {
        throw new ArgumentException("Star rating must be an integer of: 1, 2, 3, 4, 5.");
    }

    if(string.IsNullOrEmpty(userName))
    {
        throw new ArgumentException("UserName cannot be null.");
    }

    this.Stars = stars;
    this.UserName = userName;
    this.Review = review;
}
```

We now want to add more business logic to the **Item** class. We will continue to use the constructor as a place to enforce validation making it *impossible* to have invalid objects in our system!

If we think about adding a rating, this would only happen *after* the Item was created, purchased, and reviewed. We will create a method on the Item class that appends reviews.

- Add a constructor to inside the Item class:

```
public Item(string name, string description, string brand, decimal price)
{
    if (string.IsNullOrEmpty(name))
    {
        throw new ArgumentNullException(name);
    }

    if (string.IsNullOrEmpty(description))
    {
        throw new ArgumentNullException(description);
    }

    if (string.IsNullOrEmpty(brand))
    {
        throw new ArgumentNullException(brand);
    }

    if(price < 0.00m)
    {
        throw new ArgumentException("Price must be greater than zero.");
    }

    Name = name;
    Description = description;
    Brand = brand;
    Price = price;
}
```

Before we can add a method to append ratings, we need to add a ratings array to hold all customer ratings.

- Add the following property:

```
public string Brand { get; set; }
public decimal Price { get; set; }
public List<Rating> Ratings { get; set; } = new List<Rating>();
```

- Create the method to add a new rating (remember that we have constructor logic that only allows us to add valid ratings):

```
public void AddRating(Rating rating)
{
    this.Ratings.Add(rating);
}
```

We are now ready to add the API controller to handle HTTP Requests and return HTTP Responses. Recall that we *decorate* our classes and methods so that ASP.NET Web API can use the verb, headers, and body to route the request to the right class and method.

- Create the CatalogContoller class with the following code:

```
using Microsoft.AspNetCore.Mvc;
using Jet.Piranha.Domain.Catalog;

namespace Jet.Piranha.Api.Controllers
{
    [ApiController]
    [Route("[controller]")]
    public class CatalogController : ControllerBase
    {

    }
}
```

Important:

- For this class to work as a controller, it must inherit from **`ControllerBase`**
- For this class to work as a controller, it must be decorated with **`[ApiController]`**
- For the routing to work, it must have a **`Route`** attribute defined
- For this class, we are using **`[controller]`**, which maps to the route **`/catalog`**

- Create an action method to respond to HTTP GET:

```
[HttpGet]
public IActionResult GetItems()
{
    return Ok("hello world.");
}
```

Important:

- The method is decorated with [**HttpGet**], so it will only receive requests with GET verb

- Verify things are working by building the solution
  - **dotnet restore && dotnet build**
- In the terminal, navigate to the API project and run it
  - **cd YOUR-PROJECT-NAME.Api**
  - **dotnet run**
  - Verify by visiting the following URL: (your port numbers will be different)**:**
    - **http://localhost:5243/catalog**
    - **https://localhost:5244/catalog**

- Verify that the API returns “hello world.” in the browser and in the REST Client

Hint: the full URL path will be https://localhost:{YOUR_HTTPS_PORT}/catalog. This is the ASP.NET Routing engine at work (review lecture: resolve route URI, select controller class, select action method).

Once the API is returning valid responses, we will simulate calls to the database and return a hard-coded response.

- Replace the **GetItems** method with the following code:

```
[HttpGet]
public IActionResult GetItems()
{
    var items = new List<Item>()
    {
        new Item("Shirt", "Ohio State shirt.", "Nike", 29.99m),
        new Item("Shorts", "Ohio State shorts.", "Nike", 44.99m)
    };

    return Ok(items);
}
```

- Rebuild and re-run the project and open the project's Swagger page: https://localhost:{YOUR_HTTPS_PORT}/swagger

Note: Swagger is an OpenAPI implementation that magically produces documentation for the API. This is generated from the attributes defined in the class and methods. You can read more here.

- Verify the API with the Swagger page. Use it to explore and test the method.

- Verify the API in Postman (use the HTTPS link). The response should look like the following:

3. Catalog API - Get Item by Id

The API currently returns every item in the catalog. We will now add the ability to get a single item from the catalog. We want the URI to follow general REST standards, which will look like this **'/catalog/1234'**

Verb	Route	Function
GET	/catalog	Gets All Items
GET	/catalog/{id}	Gets Item by Id

This is pretty straightforward to do. Add a new method to the **CatalogController.cs** with the following code:

```
[HttpGet("{id:int}")]
public IActionResult GetItem(int id)
{
    var item = new Item("Shirt", "Ohio State shirt.", "Nike", 29.99m);
    item.Id = id;

    return Ok(item);
}
```

Important::

- Remember, actions are routed by the engine based on the verb, the route definition, and the parameter.
- The method is decorated with **`[HttpGet("{id:int}")]`**, so it receives HTTP Get requests.
- The method is decorated with **`[HttpGet("{id:int}")]`**, so it receives the 'id' from the URI: /catalog/1234
- The route *parameter* is sent to the method `GetItem(int id)`
- The route parameter is appended so the base class route and becomes `/catalog/{id}`

Verify that things are working in Swagger and in REST Client.

4. Catalog API - Post New Item

The API needs to accept new Items to the catalog.

Verb	Route	Function
GET	/catalog	Gets All Items
GET	/catalog/{id}	Gets Item by Id
POST	/catalog	Post new Item

The complexity here will come once we hook up the database. For now, just add a method with the following code:

```
[HttpPost]
public IActionResult Post(Item item)
{
    return Created("/catalog/42", item);
}
```

Important::

- Remember, actions are routed by the engine based on the verb, the route definition, and the parameter.
- The ASP.NET routing engine is looking for a route of '`/catalog`' with the POST verb and an Item in the body.

Verify that things are working in Swagger and in REST Client. The test message for POST looks like this:

```
### POST a new Item to the Catalog collection
POST https://localhost:7250/catalog
Content-Type: application/json

{
    "name": "Shoes",
    "description": "Ohio State shoes",
    "brand": "Nike",
    "price": 129.99
}
```

5. Catalog API - Post New Rating By Item Id

This step will show how child data is handled in an API. We will be updating an Item with a Rating.

Verb	Route	Function
GET	/catalog	Gets All Items
GET	/catalog/{id}	Gets Item by Id
POST	/catalog	Post new Item
POST	/catalog/{id}/ratings	Post a new rating to an Item's Ratings collection

This code will also change when we add the database. For now, we will add enough code to make the point:

```
[HttpPost("{id:int}/ratings")]
public IActionResult PostRating(int id, [FromBody] Rating rating)
{
    var item = new Item("Shirt", "Ohio State shirt.", "Nike", 29.99m);
    item.Id = id;
    item.AddRating(rating);

    return Ok(item);
}
```

Important:

- In this case, we specify additional routing in the Post verb definition: [HttpPost("{id:int}/ratings")]
- ASP.NET will automatically match the two parameters from the HTTP Request

The ASP.NET routing engine is looking for a *path* of `'/catalog/{id}/ratings'`, the POST verb, the **id** parameter in the path, and a **ratings** object in the body.

Verify that things are working in Swagger and in REST Client. Use the above test code as a guid.

6. Catalog API - Put Item By Item Id

This step will allow us to update Item using its ID as a lookup value.

Verb	**Route**	**Function**
GET	/catalog	Gets All Items
GET	/catalog/{id}	Gets Item by Id
POST	/catalog	Post new Item
POST	/catalog/{id}/ratings	Post a new rating to an Item's Ratings collection
PUT	/catalog/{id}	Put an update to an Item based on its ID

Again, skipping the database complexity for now, add the following method:

```
[HttpPut("{id:int}")]
public IActionResult Put(int id, Item item)
{
    return NoContent();
}
```

Verify that things are working in Swagger and in REST Client.

7. Catalog API - Delete Item By Item Id

This step will allow us to update Item using its ID as a lookup value.

Verb	Route	Function
GET	/catalog	Gets All Items
GET	/catalog/{id}	Gets Item by Id
POST	/catalog	Post new Item
POST	/catalog/{id}/ratings	Post a new rating to an Item's Ratings collection
PUT	/catalog/{id}	Put an update to an Item based on its ID
DELETE	/catalog/{id}	Delete an Item from the collection

Again, skipping the database complexity for now, add the following method:

```
[HttpDelete("{id:int}")]
public IActionResult Delete(int id)
{
    return NoContent();
}
```

Verify that things are working in Swagger and in the REST Client. Be sure to check every route combination from the table above.

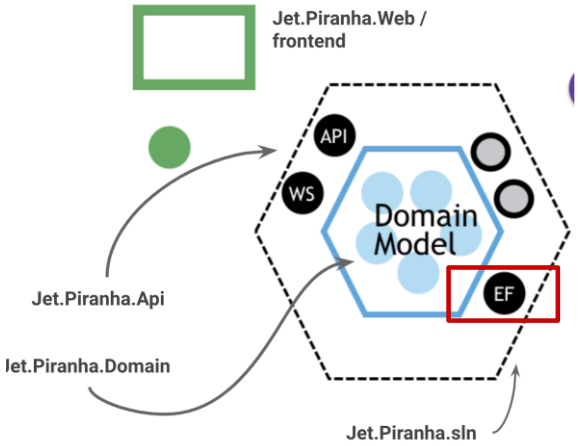
In the next lab, we will add support for a database and use code generation to create the other controllers.

Part 2 - Data Access

Currently, the API for catalog is in place and it handles HTTP Requests and Response, but it does not actually do anything. Now, we will add support for database access. We will use SQLite for local development (you may need to install SQLite), and switch to Azure when we move the application to the cloud.

1. Project Update

First, we need to update the API project to include database access. We will continue to follow the hexagonal architecture pattern:



- Create a new feature branch **ef-core**
- Create the data project. From inside the root folder, run **`dotnet new classlib -n YOUR-PROJECT-NAME.Data`**
- Add the new data project to the solution; run the command **`dotnet sln add */*.csproj`**
- To use the domain layer in the data layer, we must add a project reference from domain to data: **`dotnet add YOUR-PROJECT-NAME.Data/YOUR-PROJECT-NAME.Data.csproj reference YOUR-PROJECT-NAME.Domain/YOUR-PROJECT-NAME.Domain.csproj`**
- The API is the *runtime* for our application, so it will need a reference to the data project: **`dotnet add YOUR-PROJECT-NAME.Api/YOUR-PROJECT-`**

`NAME.Api.csproj reference YOUR-PROJECT-NAME.Data/YOUR-PROJECT-NAME.Data.csproj`

- Verify things are working by building the solution
    - `dotnet restore && dotnet build`

- Commit your changes, push to GitHub, and hand off to a teammate.

2. Create Database Context

We need to add some supporting tools and packages to help create data access. Everyone on the team should perform this step- this is a local machine installation and does not go into source control. Run the following commands:

- **`dotnet tool install --global dotnet-ef`**
- **`dotnet tool install --global dotnet-aspnet-codegenerator`**

Now, we need to install a package for the database, which will also include its child dependencies.clear

- Inside *both* the **API** and **Data** project folders, add the SQLite package
  - **`dotnet add package Microsoft.EntityFrameworkCore.Sqlite`**

- Rename the **Class1.cs** file to **StoreContext.cs** and give it the following code (be sure to use your projects code name, not **Jet.Piranha**):

```
using Jet.Piranha.Domain.Catalog;
using Microsoft.EntityFrameworkCore;

namespace Jet.Piranha.Data
{
    public class StoreContext : DbContext
    {
        public StoreContext(DbContextOptions<StoreContext> options)
            : base(options)
        { }

        public DbSet<Item> Items { get; set; }
    }
}
```

Note: we added a default constructor for **StoreContext** that calls the constructor of the parent base class **DbContext**. We also added a **DbSet** to represent **Items** in our catalog.

Verify things build, commit your changes, push to GitHub, and hand off to a teammate.

3. Configure the API

The API is our project's *runtime;* it holds all the required resources to start a .NET project and manage it in the CLR. The Domain and Data projects are simple class libraries ***used*** by the API.

We must configure the runtime to include the **StoreContext** as part of the application. It will use the ASP.NET Middleware (see lecture on 'Middleware') to provide **StoreContext** as a shared service.

Edit the **Startup.cs** class's ConfigureServices() method to include the StoreContext and its dependencies:

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddControllers();
    services.AddEndpointsApiExplorer();
    services.AddSwaggerGen();
    services.AddDbContext<StoreContext>(options =>
        options.UseSqlite("Data Source=../Registrar.sqlite",
            b => b.MigrationsAssembly("jet.piranha.Api")));
}
```

Remember to include **`using YOUR-PROJECT-NAME.Data`** in this file.

For now, we will use SQLite for all local development. When we migrate the application to the cloud, we will swap this out for Azure.

Now, let's update the **CatalogController** to use the StoreContext. We are going to use an off-the-shelf feature in ASP.NET called dependency injection to get the StoreContext from the services we just configured.

- Inside CatalogController.cs, add a new private variable to hold the store context:
- Add a new constructor that accepts an instance of StoreContext and sets it to the local variable.

```
using Microsoft.AspNetCore.Mvc;
using Emerald.Tiger.Domain.Catalog;
using Emerald.Tiger.Data;

namespace Emerald.Tiger.Api.Controllers
{
    [ApiController]
    [Route("/catalog")]
    public class CatalogController : ControllerBase
    {
        private readonly StoreContext _db;

        public CatalogController(StoreContext db)
        {
            _db = db;
        }
```

ASP.NET will recognize that the CatalogController needs a StoreContext, and everytime it instantiates the controller, it adds the context- it's that simple.

Update the GetItems method to return all items for the API collection / database table:

```
[HttpGet]
public IActionResult GetItems()
{
    return Ok(_db.Items);
}
```

Run the API project and use Swagger or REST Client to call **`GET /catalog`**. You should receive the following error:

```
The entity type 'Rating' requires a primary key to be defined. If you intended to use a keyless entity type, call 'HasNoKey' in 'OnModelCreating'.
```

This is because when we created the initial **Rating** class, we didn't give it an Id property. We have two choices here- follow the convention or write custom configuration code for `OnModelCreating`. Recall from the lecture that we want to follow convention where possible.

In the **Domain** project, open the **Rating** class and add an **Id** property:

```
public class Rating
{
    public int Id { get; set; }
    public int Star { get; set; }
```

Rerun the project and test again. This time, you should get a new error:

```
SQLite Error 1: 'no such table: Items'.
```

EF Core expects to have an existing database to connect to. Note that it *did* create a .db file inside the root folder. Our issue is that we do not have a database schema with tables and data. We will build that next.

Before checking the code in and handing off to a teammate, update the *.**gitignore** file to exclude the database- we should never check databases into GitHub.

```
# exclude database files
*.sqlite
```

Commented [PK1]: We chose database filename Registrar.sqlite

4. Adding Startup Data

Adding seed data requires two steps: creating an initializer, and configuring it to optional run when the program starts.

Inside the **Data** project, create a **DbInitializer.cs** file with the following code:

```
using Emerald.Tiger.Domain.Catalog;
using Microsoft.EntityFrameworkCore;

namespace Emerald.Tiger.Data
{
    public static class DbInitializer
    {
        public static void Initialize(ModelBuilder builder)
        {
            builder.Entity<Item>().HasData(
                new Item("Shirt", "Ohio State shirt", "Nike", 29.99M)
                {
                    Id = 1
                },
                new Item("Shorts", "Ohio State shorts", "Nike", 44.99m)
                {
                    Id= 2
                }
            );
        }
    }
}
```

Inside the **StoreContext** class, override the base class's `OnModelCreating` method to seed the data:

```
public DbSet<Item> Items { get; set; }

protected override void OnModelCreating(ModelBuilder builder)
{
    base.OnModelCreating(builder);
    DbInitializer.Initialize(builder);
}
```

From the terminal window navigate to the **Api** project folder and add the following package reference:

- `dotnet add package Microsoft.EntityFrameworkCore.Design`

EF Core contains a tool to publish and deploy database changes. Next we need to run the following command to create a migration:

- `dotnet ef migrations add init`

You should see a new folder inside the **Api** project called **Migrations**. You can open the code to see what EF core generated- but no touching!

To publish the changes to the database, run the following command:

- `dotnet ef database update`

You should see a bunch of text fly by- those are the SQL commands that EF Core is writing and executing on your behalf.

Verify things build, and that the API now returns data from the database. When done, commit your changes, push to GitHub, and hand off to a teammate

5. Adding Get By Id

The hard problems are now solved; all that's left is removing all of the hard-coded API responses and replacing them with the **StoreContext**.

Replace the **GetItem** method body with the following:

```
[HttpGet("{id:int}")]
public IActionResult GetItem(int id)
{
    return Ok(_db.Items.Find(id));
}
```

Run and test the API; use id's of 1, 2, and 3. The first two should return fine, but the third one will give you a '204'. This is not what we want or expect; it should be a '404'.

Update the **GetItem** method to check if the item is found:

```
[HttpGet("{id:int}")]
public IActionResult GetItem(int id)
{
    var item = _db.Items.Find(id);
    if (item == null)
    {
        return NotFound();
    }
    return Ok();
}
```

Verify things build, and that the API now returns 404 for unfound items. When done, commit your changes, push to GitHub, and hand off to a teammate.

5. Adding Post

Post is pretty easy, update the Post method as follows:

```
[HttpPost]
public IActionResult Post(Item item)
{
    _db.Items.Add(item);
    _db.SaveChanges();
    return Created($"/catalog/{item.Id}", item);
}
```

Testing a POST is a little more difficult, because you have to know how to construct the HTTP Request Body. In Swagger or REST Client, use the following as the body for your test:

```
{
  "name": "Shoes",
  "description": "Running Shoes",
  "brand": "Nike",
  "price": 109.99
}
```

Be sure to test the unhappy path too- try sending a negative price or a body with no name. These requests get rejected *before* they hit our API due to the logic we built into the domain layer. The correct thing to do is to catch the error and return a “400”, but that is a little involved and out of scope for this course.

When done, commit your changes, push to GitHub, and hand off to a teammate.

7. Adding Post Rating

Posting a Rating isn't hard, but we have to think a little bit about creating the logic. We already have a method in our domain for adding a rating to an item. In our API, we will need to find the item, add the rating, and save it back to the database. That code looks like this:

```
[HttpPost("{id:int}/ratings")]
public IActionResult PostRating(int id, [FromBody] Rating
rating)
{
    var item = _db.Items.Find(id);
    if (item == null)
    {
        return NotFound();
    }

    item.AddRating(rating);
    _db.SaveChanges();

    return Ok(item);
}
```

Look at how we tested the Post for Item and follow that pattern to test for adding a Rating. Verify that things build, commit your changes, push to GitHub, and hand off to a teammate.

8. Adding Put

Put is also simple but requires using some arcane features of EF Core. Update the Put method with the following code:

```
public IActionResult PutItem(int id, [FromBody] Item item)
{
    if (id != item.Id)
    {
        return BadRequest();
    }

    if (_db.Items.Find(id) == null)
    {
        return NotFound();
    }

    _db.Entry(item).State = EntityState.Modified;
    _db.SaveChanges();

    return NoContent();
}
```

The first few blocks should make sense. We are using EF Core features Entry and EntityState to load an item and overwrite it with a new item. You will need to use the 'lightbulb' to add a reference to EntityFramework.

9. Adding Delete

Delete is also very easy to do, update its method with the following:

```
[HttpDelete("{id:int}")]
public IActionResult DeleteItem(int id)
{
    var item = _db.Items.Find(id);
    if (item == null)
    {
        return NotFound();
    }

    _db.Items.Remove(item);
    _db.SaveChanges();

    return Ok();
}
```

Test each of the methods together- verify that you can create, retrieve, update, and delete items at will.

10. Generating OrdersController

To show the power of ASP.NET and EF Core, we are going to rapidly generate a controller for our orders.

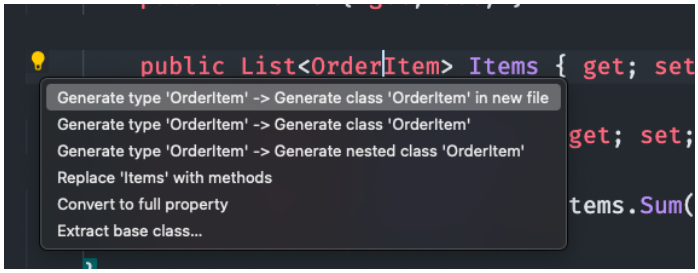
Create a new **Orders** folder in the **Domain**, along with an **Order** class:

```
using System.Collections.Generic;
using System.Linq;

namespace Jet.Piranha.Domain.Orders
{
    public class Order
    {
        public int Id { get; set; }
        public List<OrderItem> Items { get; set; }
        public DateTime CreatedDate { get; set; }
        public decimal TotalPrice => Items.Sum(i => i.Price);

    }
}
```

Use the 'lightbulb' to generate a new class for **OrderItem**:



Give **OrderItem** the following code:

```
using Jet.Piranha.Domain.Catalog;

namespace Jet.Piranha.Domain.Orders
{
    public class OrderItem
    {
        public int Id { get; set; }
        public Item Item { get; set; }
        public int Quantity { get; set; }
        public decimal Price => Item.Price * Quantity;
    }
}
```

In the **Data** project, add a new **Orders DbSet** to the context:

```
public class StoreContext : DbContext
{
    public StoreContext(DbContextOptions<StoreContext> options)
        : base(options)
    { }

    public DbSet<Item> Items { get; set; }

    public DbSet<Order> Orders { get; set; }
}
```

In the Data project, add StoreContextFactory.cs.
(Note: use your project name on line 4).

```
1   using Microsoft.EntityFrameworkCore;
2   using Microsoft.EntityFrameworkCore.Design;
3
4   namespace jet.piranha.Data;
5
    0 references
6   public class StoreContextFactory : IDesignTimeDbContextFactory<StoreContext>
7   {
        0 references
8       public StoreContext CreateDbContext(string[] args)
9       {
10          var optionsBuilder = new DbContextOptionsBuilder<StoreContext>();
11
12          optionsBuilder.UseSqlite("Data Source=../Registrar.sqlite");
13
14          return new StoreContext(optionsBuilder.Options);
15      }
16  }
```

Back in the API project, we need to add some package dependencies for this trick to work. Run the following commands inside the API project:

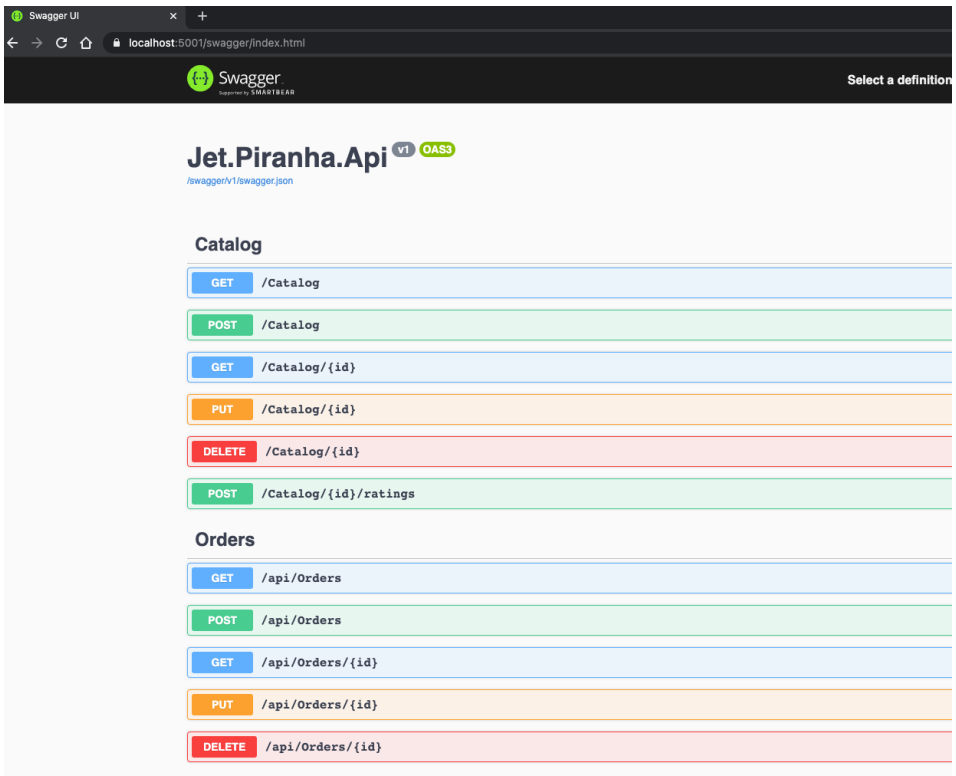
- **dotnet add package Microsoft.VisualStudio.Web.CodeGeneration.Design**
- **dotnet add package Microsoft.EntityFrameworkCore.Design**
- **dotnet add package Microsoft.EntityFrameworkCore.SqlServer**
- **dotnet add package Microsoft.EntityFrameworkCore.Tools**

Finally, we are ready to generate our OrdersController based on our StoreContext combined with ASP.NET and EF Core conventions.

- **dotnet aspnet-codegenerator controller -name OrdersController -async -api -m Order -dc StoreContext -outDir Controllers -api**

Open the new **OrdersController** and inspect the code that was generated. This code used asynchronous methods, which is a little bit different than our **CatalogController**.

Run the API application, and verify that the Swagger file shows the newly generated methods:



For the lab submission, submit the link to your API repository on GitHub.